Компьютерные науки и анализ данных

Yandex

Faculty of Computer science
Higher School of Economics

Программирование на Python

лектор: Сергей Горшков

План

- Генераторы, итераторы и iterable-объекты
- Методы генераторов
- Знакомство с collections: именованные кортежи, деки
- Аннотации функций
- Декораторы. Параметрические декораторы

Так в чем же отличие итераторов и генераторов???

- Итератор – более общая концепция, чем генератор.
- Итератор – это интерфейс доступа к элементам коллекций и потоков данных. Он требует реализации единственного метода – «дай мне следующий элемент». Если вы пишите свой итератор на Python 3 вам нужно реализовать в классе метод __next__. Если элементы исчерпаны итератор возбудит исключение StopIteration.
- Генератор – это итератор, но не наоборот. Не любой итератор является генератором.
- Нам незачем самим определять методы __iter__() и __next__(), так как они неявно присутствуют у генератора.
- Как взять из итератора (в т. ч. из генератора) N первых значений? itertools.islice (pyc)

Есть же еще и iterable!

- От итерируемого объекта можно получить его "копию"-итератор; генератор является разновидностью итератора.
- У итерируемого объекта, то есть объекта, который можно "превратить" в итератор, должен быть метод __iter__(), который возвращает соответствующий объект-итератор.
- В некоторых источниках итератор рассматривается как частный случай итерируемого объекта, поскольку оба поддерживают операцию итерации, то есть обход циклом for. Однако for работает только с итераторами. Переданный на обработку объект должен иметь метод __iter__(), который for неявно вызывает перед обходом. Метод __iter__() должен возвращать итератор.

Всё ещё об iterable!

- У итерируемого объекта не обязательно должен быть метод __next__(), который используется при итерации.
- У итератора обязан быть метод __next__(), извлекающий из итератора следующий элемент.
- У итераторов, так же как у итерируемых объектов, есть метод __iter__(). Однако для итератора он возвращает сам объект-итератор
- Примеры итерируемых объектов в Python - список, словарь, строка, range.
- Примеры итераторов - файловые объекты, генераторы, итераторы созданные на основе списка, строки, объекта типа range и т. д.
- В Python есть встроенные функции iter() и next(), которые соответственно вызывают методы __iter__() и __next__() объектов, переданных в качестве аргумента.

А как с этим работает for?

- Внутренний механизм цикла for сначала вызывает метод __iter__() объекта. Так что, если передан итерируемый объект, создается итератор. После этого применяется метод __next__() до тех пор, пока не будет возбуждено исключение StopIteration.
- Поскольку метод __iter__() итератора возвращает сам итератор, то после перебора циклом for объект исчерпывается. То есть получить данные из итератора можно только один раз.
- В случае с коллекциями это не так. Здесь создается другой объект - итератор. Он, а не итерируемый объект, отдается на обработку циклу for.

Генераторы

- `a = (x for x in range(5) if x % 2 == 1)` – *генераторное выражение*
- Бесконечный генератор: while True, внутри которого yield – *генераторная функция*
- Метод next – берёт следующее значение из генератора
- `throw()` – бросить исключение внутри генератора. Исключение будет возбуждено из того выражение yield, где генератор последний раз остановился.
- `close()` – закрыть генератор. Бросает внутри генератора особое исключение GeneratorExit. После этого `next()` будет возвращать StopIteration

Передача значений в генератор

- Передача значений в генератор – `send` – отправляем значения на обработку
- Первый раз нельзя посылать в генератор данные, пока мы не прокрутили его до первого yield. Нужно либо взывать `next(g)` или `g.send(None)` – это одно и тоже.
-
```
def generator(x):
    while True:
        x = yield x + 1
```
- g = generator(5)
- g.send(None)
- g.send(10)

Именованные кортежи

- Модуль collections, namedtuple

  ```
  from collections import namedtuple
  Car = namedtuple('Car' , 'color mileage')
  my_car = Car('red', 3812.4)
  my_car.color
  ```

- Распаковка кортежа и оператор * сработают как обычно для my_car, можно обращаться и по индексу
- дек

Аннотации типов

- Аннотации пишут прямо в коде, что ломает обратную совместимость
- Аннотации используются IDE и программистом, но не интерпретатором!
- В простейшем случае аннотация содержит ожидаемый тип
- Аннотации для переменных пишут через двоеточие после идентификатора
  - price: int = 5 # с последующей инициализацией
  - title: str
  - title = 5 # здесь ошибки не будет при работе интерпретатора!
- Параметры функции аннотируются так же как переменные, а возвращаемое значение указывается после стрелки -> и до завершающего двоеточия.
  - def indent_right(s: str, width: int) -> str:
  - return " " * (max(0, width - len(s))) + s

Модуль typing

- Расширение стандартных типов. Используются `[ ]`
- Optional[type]: указанный тип или None
  - price: Optional[int] # II
- Any – любой тип (в отличие от object, который определяет множество методов)
- Union – использование типов из множества. Optional[T] эквивалентна Union[T, None]
  - Union[int, float]
- Коллекции: иногда недостаточно указывать list, tuple и проч., а надо конкретизировать тип элементов в коллекции

Модуль typing. Коллекции

- Список однотипных элементов : List[type].
- Аналогично с гомогенными кортежами: prices: Tuple[int, ...] = (1, 2)
- Аннотация Tuple без указания типов элементов работает аналогично Tuple[Any, ...]
- Многоточие означает множество элементов
- Можем задать разные типы для элементов с разными индексами
  - price_with_title: Tuple[int, str] = (1, "hello")
- Генератор с yield будет возвращать Iterable какого-то типа
  - def generate_two() -> Iterable[int]:
  - yield 1
- Если функция не возвращает ничего, то для аннотации следует использовать None:
  - def nothing(a: int) -> None:

Декораторы. Лирика

- Пререквизиты:
  - Функции в python являются объектами 1 класса, соответственно, их можно возвращать из другой функции или передавать в качестве аргумента в функцию.
  - В питоне можно определять вложенные функции
- Декораторы — это, по сути, "обёртки", которые дают нам возможность изменить поведение функции, не изменяя её код.
- Один и тот же декоратор можно применить к разным функциям
- К одной функции можно применить несколько декораторов последовательно

Декораторы

- ```
  def decorator(function_to_decorate):
      def the_wrapper_around_the_original_function():
          print("Before")
          res = function_to_decorate()
          print("After")
          return res
      return the_wrapper_around_the_original_function
  def our_function():
      print("Hello")
  our_function_decorated = decorator(our_function)
  ```
- Если хотим, чтобы всегда вызывалась функция после декорирования, то перезапишем её: `our_function = decorator(our_function)`

Higher School of Economics
Faculty of Computer science
  ```

Декораторы

- Или же пользуемся собственно декораторами в Python:
-
```
@decorator
def our_function():
    print("Hello")
```
- Декораторы в python — это просто синтаксический сахар для конструкций вида:
  - `our_function = decorator(our_function)`

Декораторы для функций с аргументами

```
def decorator(function_to_decorate):
    def the_wrapper_around_the_original_function(*args, **kwargs):
        print("Before")
        res = function_to_decorate(*args, **kwargs)
        print("After")
        return res
    return the_wrapper_around_the_original_function

@decorator
def our_function(*args, **kwargs):
    print(args, kwargs)
# our_function=decorator(our_function)
our_function("Hello", arg="Arg")
```

Параметрические декораторы

```
def decorator_maker_with_arguments( *decorator_args):
    def my_decorator(func):
        def wrapped(*function_args):
            print(*decorator_args, *function_args)
            return func(function_args)
        return wrapped
    return my_decorator

@decorator_maker_with_arguments("параметры",  "декоратора")
def decorated_function_with_arguments(function_args):
    print (function_args)

decorated_function_with_arguments("параметры ", "функции")
```

Декоратор @functools.wraps

- Декоратор копирует всю информацию об оборачиваемой функции (её имя, из какого она модуля, её документацию и т.п.) в функцию-обёртку.
- Тогда будут корректно будет работать __name__, __module__, __doc__
- from functools import wraps
- @wraps(func)
- Запоминаем имя функции в wraps перед вызовом обертки внутри функции, в которую передана вызываемая func (в наших примерах `wrapped`)
- К слову, в функциях может быть docstring. Это строка, описывающая что делает функция, следует в тройных кавычках на следующей строке после прототипа функции перед её телом. Вызов его: __doc__

Faculty of Computer Science
Higher School of Economics

sgorshkov@hse.ru
tg: @Seragor98